

Санкт-Петербургский государственный университет

Системного программирования

Группа 24.V71-мм

# Разработка фреймворка анализа повторов в документации в рамках проекта DocLine

*Макаров Павел Михайлович*

Отчёт по преддипломной практике  
в форме «Производственное задание»

Научный руководитель:  
доцент кафедры системного программирования, к.ф.-м.н., Луцив Д.В.

Санкт-Петербург  
2026

# Оглавление

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                                   | <b>3</b>  |
| <b>1. Анализ предметной области</b>                                               | <b>6</b>  |
| 1.1. Обзор основных алгоритмов . . . . .                                          | 6         |
| 1.2. Обзор существующих технических решений . . . . .                             | 8         |
| 1.3. Duplicate Finder . . . . .                                                   | 11        |
| 1.4. Язык и экосистема Golang . . . . .                                           | 13        |
| 1.5. Возможности больших языковых моделей в задаче анали-<br>за текстов . . . . . | 14        |
| <b>2. Требования к фреймворку</b>                                                 | <b>16</b> |
| 2.1. Случаи использования и функциональные требования .                           | 16        |
| 2.2. Нефункциональные требования . . . . .                                        | 17        |
| <b>3. Архитектура фреймворка</b>                                                  | <b>19</b> |
| 3.1. Структура фреймворка . . . . .                                               | 19        |
| 3.2. Шаблон использования . . . . .                                               | 22        |
| <b>4. Особенности реализации</b>                                                  | <b>24</b> |
| 4.1. Система модулей . . . . .                                                    | 24        |
| 4.2. Встроенные алгоритмы . . . . .                                               | 26        |
| 4.3. Обработка документов . . . . .                                               | 29        |
| 4.4. Создание отчётов . . . . .                                                   | 31        |
| 4.5. Открытый API . . . . .                                                       | 33        |
| <b>5. Тестирование и апробация</b>                                                | <b>36</b> |
| 5.1. Тестирование фреймворка . . . . .                                            | 36        |
| 5.2. Апробация фреймворка . . . . .                                               | 37        |
| <b>Заключение</b>                                                                 | <b>38</b> |
| <b>Список литературы</b>                                                          | <b>39</b> |

# Введение

Документация является неотъемлемой частью сложных проектов. В процессе развития программного обеспечения объём текстовой документации имеет тенденцию к увеличению, смысловое содержание при этом может становиться запутанным и противоречивым [5, 15, 34]. Вследствие этого, на этапах разработки возрастает вероятность возникновения ошибок и неточностей, связанных с неактуальным или некорректным описанием функциональности. Неудовлетворительное ведение документации способно существенно повысить расходы на разработку продукта и на обучение новых членов команды разработчиков [7, 37, 38].

В крупных проектах документация фактически разрабатывается параллельно несколькими командами или отдельными специалистами, каждый из которых работает над определёнными частями продукта. Подобная организация труда без организации связи между разработчиками неизбежно приводит к несоответствиям в описании схожих терминов, функций и прочих компонентов. Из-за этого поддержание документации в актуальном и корректном состоянии превращается в отдельную инженерную задачу [20].

Одним из решений указанной проблемы является применение специализированных программных инструментов для создания и сопровождения документации [10, 12, 25]. Примером такого подхода служит проект DocLine, разрабатываемый на кафедре системного программирования Санкт-Петербургского государственного университета [8, 16, 26]. Целью проекта является создание метода повторного использования документации, а также подхода к поиску и анализу повторяющихся фрагментов в документационных текстах. В рамках проекта DocLine были реализован программный инструмент Duplicate Finder, предназначенный для анализа повторов текста в документации.

Среди значимых разработок в данной области следует выделить DocBook и DRL (Documentation Reuse Language). DocBook представляет собой открытый стандарт подмножества языков разметки, ориен-

тированный на создание технической документации, однако допускающий также формирование разметки для книг, статей, презентаций и веб-сайтов. DRL, в свою очередь, является языком, предназначенным для проектирования и разработки документации с акцентом на повторное использование.

Инструмент Duplication Finder представляет собой легковесный автономный инструмент для обнаружения клонов (текстовых повторов) в документации. Он использует внешние средства для поиска как чётких, так и нечётких повторов текста. Duplication Finder сочетает в себе код на языках Java и Python. В качестве основного формата представления и рефакторинга документации используется язык DRL, что предоставляет возможности для анализа и улучшения больших объёмов технической документации. Данная разработка позволяет обнаруживать дублирующиеся фрагменты в файлах документации и преобразовывать их в переиспользуемые информационные элементы.

История развития проекта DocLine идёт с 2007 года [19, 33]. В процессе разработки программных инструментов применялось несколько языков программирования для реализации различного функционала. Однако мультязычность реализации порождает ряд проблем, характерных для долгосрочной разработки программного обеспечения, особенно в условиях университетской разработки, когда множество людей на протяжении большого времени реализовывало программный продукт. В первую очередь к таким проблемам относятся сложности эксплуатации. Например, для запуска Duplicate Finder необходимо установить несколько сторонних программных компонентов (JRE для Neuristic Finder, .NET Framework для Fuzzy Finder), а также выполнить ряд дополнительных условий (наличие архитектуры x86\_64 для запуска Clone Miner, использование имени компьютера в сети, состоящего только из символов ASCII, и другие). Указанные ограничения существенно затрудняют как использование инструмента, так и его последующую поддержку.

Ввиду выявленных проектных просчётов и недостатков существующего инструментария была осознана необходимость изменений для

сосредоточения усилий на создание чётко ограниченного по функциональности инструмента разработки программного обеспечения. Возникла потребность в разработке нового программного продукта для поиска нечётких повторов в тексте с возможностью последующего расширения функционала по мере необходимости.

Для создания обновлённого инструмента поиска повторов в тексте требуются активно развивающиеся и актуальные технологии, обладающие высокой отказоустойчивостью. В качестве основного языка программирования для решения указанных задач целесообразно выбрать язык Go. Данный язык был разработан с акцентом на производительность, высокую эффективность как на этапе компиляции, так и при выполнении кода, а также на простоту написания надёжных и масштабируемых программ.

Целью данной дипломной работы является создание модульного фреймворка для обеспечения технологической актуальности и практичности использования алгоритмов поиска повторов в программной документации. Для достижения этой цели в рамках работы были сформулированы следующие задачи.

- Проанализировать область поиска дубликатов в программной документации, найти алгоритмы и готовые решения.
- Сформулировать требования в фреймворку.
- Выработать архитектуру фреймворка.
- Выполнить реализацию фреймворка.
- Провести модульное тестирование новой реализации существующих алгоритмов работы с повторами и новых модулей поиска повторов; провести апробацию созданного инструмента на основе документации существующих программных проектов.

# 1. Анализ предметной области

## 1.1. Обзор основных алгоритмов

Технология распознавания повторов в тексте активно начала развиваться с начала 90-х годов [3, 36]. Результатом этого стали различные программные инструменты, упрощающие, автоматизирующие процесс поиска повторов.

Одним из примеров алгоритмов поиска нечётких повторов является **алгоритм шинглов** (w-shingling) [24]. Алгоритм сравнивает случайную выборку контрольных сумм шинглов (подпоследовательностей) двух текстов между собой, что позволяет определить их сходство или различие. [21]

Основное использование алгоритма шинглов - системы антиплагиата для выявления оригинальности текстов. Алгоритм позволяет сравнивать проверяемую работу с другими текстами из базы данных различных исследований, журналов, статей и т.д для выявления совпадающих фрагментов.

Вычисление шинглов происходит следующим образом: пусть документ  $D = (t_1, t_2, \dots, t_n)$  — последовательность токенов (символов или слов). Множество **k-шинглов** документа определяется как:

$$S(D, k) = \{(t_i, t_{i+1}, \dots, t_{i+k-1}) \mid i = 1, 2, \dots, n - k + 1\}.$$

Для двух документов  $A$  и  $B$  их сходство вычисляется через коэффициент Жаккара для множеств шинглов:

$$J(A, B) = \frac{|S(A, k) \cap S(B, k)|}{|S(A, k) \cup S(B, k)|}.$$

**Сходство Жаккара** — метрика сходства двух последовательностей данных. Математически сходство Жаккара определяется как отношение мощности пересечения двух множеств к мощности их объединения:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

, где  $A$  и  $B$  — два множества,  $|A \cap B|$  — мощность пересечения множеств, а  $|A \cup B|$  — мощность их объединения [27].

Основное использование сходства Жаккара - алгоритмы кластеризации и задачи квалификации данных, рекомендательные системы и поиск дубликатов в наборе данных. Также часто используется в экологии (сравнение видового состава), геномике (сравнение генетических последовательностей) и обработке естественного языка (выявление релевантных предложений).

**Сходство Левенштейна** — это метрика, измеряющая количество операций, необходимых для преобразования одной строки в другую. Операции включают вставку, удаление и замену символов. Формула для вычисления расстояния Левенштейна между двумя строками  $a$  и  $b$  длиной  $|a|$  и  $|b|$  соответственно определяется рекурсивно следующим образом:

$$d(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0 \\ \min \begin{cases} d(i-1, j) + 1 \\ d(i, j-1) + 1 \\ d(i-1, j-1) + cost \end{cases} & \text{otherwise} \end{cases}$$

где  $cost$  равен 0, если символы  $a[i-1]$  и  $b[j-1]$  совпадают, и 1 в противном случае [39].

**Алгоритм Рабина–Карпа** — метод, основанный на хэшировании скользящих окон текста [13]. Он быстро находит точные или приближённые совпадения подстрок, вычисляя хэш-функции для  $k$ -символьных окон и сравнивая их с хэшами паттерна. Для текста  $T$  длиной  $n$  и паттерна  $P$  длиной  $m$  процесс выглядит так: предварительно вычисляется хэш  $h_P$  для  $P$ , затем скользящие хэши  $h_T(i)$  для окон  $T[i \dots i+m-1]$ . Если  $h_T(i) = h_P$ , проверяется точное совпадение символов, чтобы избежать коллизий. Этот метод эффективен (средняя сложность  $O(n+m)$ ) и часто комбинируется с шинглингом в антиплагиатных системах вроде Turnitin.

В статье "Detecting the origin of text segments efficiently" предложен

**алгоритм поиска ранних текстовых фрагментов** [6]. В задаче определения источников алгоритму задаётся набор  $S$  документов, упорядоченных по времени создания, и документ запроса  $D$ . Ему необходимо вывести для каждой последовательной последовательности из  $k$  буквенно-цифровых терминов в  $D$  самый ранний документ в  $S$ , в котором эта последовательность появилась (если такой документ существует).

**Алгоритм Укконена** — эффективный метод построения суффиксного дерева для строки длиной  $n$  за линейное время  $O(n)$  [29]. Он позволяет быстро находить все вхождения подстроки, наибольшие общие подстроки и решать задачи сравнения строк, разбивая строку на всевозможные комбинации суффиксов в виде дерева. Алгоритм можно использовать и для поиска повторов — строится суффиксное дерево строки за  $O(n)$ , что идеально для поиска повторяющихся подстрок и сравнения текстов. Он разбивает строку на суффиксы, храня их компактно через интервалы на рёбрах, и использует активную точку с правилом остатка для онлайн-построения. Для сравнения двух строк  $A$  и  $B$  строим дерево для  $A\#B\$$ ; глубина  $LCA$  (наименьшего общего предка) суффиксов из  $B$  даёт длину наибольшей общей подстроки (например, для "abc" и "xabcd" результат равен 3). Метод показывает себя эффективнее шинглов для больших текстов.

## 1.2. Обзор существующих технических решений

**dupeGuru** - это инструмент для поиска дублирующих файлов на компьютере, с возможностью сканирования как названий файлов, так и их содержимого с помощью алгоритма нечёткого сопоставления (fuzzy matching) [31], то есть программа будет находить повторы, даже если они не совсем схожи. Приложение работает на Windows, Linux, Mac OS X.

Несмотря на то, что основное предназначение программы - поиск дубликатов, ищет повторы программа в различных типах файлов, а не только в тексте. Это хоть и универсально, но размывает основной



функционал в целях унификации с разными видами программных файлов. Для работы используются другие алгоритмы, которые могут пригодиться в работе, но отводят нас на более глубокий уровень данных, а не на текстовый поиск повторов

**Антиплагиат** - это российская система проверки текста на заимствования [35]. Система нацелена на проверку плагиата в научных текстах (дипломы, статьи и т.д.).

Несмотря на то, что проект нацелен на поиск повторов, основное отличие в том, что поиск дубликатов проходит в других, уже существующих текстах, а не в самом анализируемом тексте. Кроме того, у системы есть зарегистрированный "Модуль поиска заимствованных изображений", что является дополнением к уже существующему текстовому поиску дубликатов.

**Trados Studio** - это программа, среда перевода на базе искусственного интеллекта, разработанная для переводчиков [28]. ПО в режиме анализа проекта может экспортировать список часто повторяющихся фраз.

Данная программа является профессиональным инструментом для переводчиков. Причём среди функционала есть функция поиска повторов, причём в отличие от двух предыдущих решений (Антиплагиат, Text.ru) ищущая плагиат именно в анализируемом тексте. Тем не менее, данный инструмент имеет лишь крайне ограниченный функционал поиска повтора, без возможности изменения настроек и алгоритма поиска повторов.

**Lingo4G** - это движок для исследования и визуализации огромного количества документов и текста. Система позволяет объединить компоненты обработки Lingo4G в аналитические запросы для выполнения различных задач интеллектуального анализа текста, начиная от простого поиска документов на основе запросов, кластеризации и 2d-отображения и заканчивая выявлением дублирующихся или похожих по содержанию документов [18].

Система имеет огромные возможности для визуализации текстовых файлов и поиска повторов. Тем не менее, возможности анализа повто-

ров ограничены лишь поиском схожил или одинаковых документов. Алгоритм не выявляет схожести одного анализируемого текста, а ищет совпадения между текстами в разных текстовых файлах.

**Natural Language Toolkit** — набор библиотек и программ для символической и статистической обработки естественного языка (NLP) [23]. Инструменты поиска повторов в NLTK: Поиск в контексте (Concordance Search), TGrep Search (Поиск по синтаксическим деревьям), WordNet Search (Семантический поиск), TokenSearcher (Поиск по токенам), ContextIndex (Поиск по контексту), Concordance App (Поиск с помощью Конкорданса). Каждый инструмент оптимизирован для конкретных задач лингвистического анализа и может использоваться независимо или в комбинации с другими компонентами библиотеки.

**NiCad Clone Detector** — гибкая гибридная система обнаружения клонов на основе TXL, чувствительная к языку и сравнению текста, использует кэширование для оптимизации повторных анализов, избегая повторного извлечения неизмененных файлов [4]. Разработан для реализации гибридного метода обнаружения клонов NiCad (Automated Detection of Near-Miss Intentional Clones) в удобном, простом в использовании инструменте командной строки. Может быть легко встроен в IDE и другие среды. Он принимает в качестве входных данных исходный каталог или каталоги, которые необходимо проверить на наличие клонов, и конфигурационный файл, в котором указываются требуемые нормализация и фильтрация, результаты выводятся в XML-формате с информацией о файлах, номерах строк и в HTML-отчеты для удобства просмотра результатов.

**DocReuse** — детектор дублирования документации, основанный на Colibri-Java-библиотеке. Основан на Анализе формальных понятий (FCA) с преобразованием данных на естественном языке в формальный контекст с объектами и атрибутами для построение решеток (lattice construction) и генерация понятий (concept generation). Результатом будет файл с клонами как концептами с минимум 2 объектами и хотя бы 1 общим атрибутом [9].

**CloneDetective** — платформа с открытым исходным кодом и це-

почка инструментов для обнаружения клонов, которая особенно ориентирована на конфигурируемость и расширяемость и, таким образом, поддерживает подготовку и проведение исследований по обнаружению клонов [11].

Проанализировав существующие решения, становится очевидно, что абсолютное большинство решений сосредоточено на поиске повторов в коде, поиске плагиата для научных целей и для поиска одинаковых файлов на жёстких дисках компьютеров. Кроме того, в этих решениях не хватает простоты использования, в них не хватает инструмента, который можно использовать в коде без лишних проблем.

### 1.3. Duplicate Finder

Duplicate Finder — легковесный инструмент для поиска клонов и рефакторинга текста документации в рамках проекта Docline. С его помощью пользователь имеет возможность находить повторы в DRL-файлах с преобразованием таких фрагментов в переиспользуемые информационные элементы [16].

Входом является файл *element\_miner\_ui.py*. После запуска отображается полноценный графический интерфейс с выбором необходимого режима поиска клонов и его настройками (минимальную и максимальную длину искомых клонов, способ проверки и восстановления нарушений XML-разметки и прочие дополнительные опции поиска в зависимости от выбранного режима), в том числе путь к анализируемому файлу. Архитектура системы состоит из нескольких модулей: ядро анализа находится в файле *clones.py* с основными алгоритмами поиска клонов и классами CloneGroup, ExactCloneGroup, FuzzyCloneGroup; в файле *clones2html.py* находится код для генерации HTML-отчётов после анализа. Пользовательский интерфейс реализован в *element\_miner\_ui.py* с помощью PyQt, для обработки файлов используется файл *pandoc\_importer.py* для конвертации различных форматов в DRL.

С использованием сторонней утилиты pandoc, разработка поддер-

живает на вход различные форматы с помощью конвертации: .drl, .xml, .docbook, .tpl для DRL-файлов; .txt, .md, .rst для текстовых форматов; .docx, .tex, .html для форматов документов. Конвертация проходит через определение формата по расширению файла, конвертация с помощью pandoc в DocBook или в текстовые данные, оборачивание в структуру DRL при необходимости.

Инструмент использует четыре основных режима анализа клонов: Clone Finder или automatic (через *clones.exe* ищет точные повторы с параметром минимальной длины фрагмента); Fuzzy Heat или interactive (создает тепловые карты и нечеткие дубликаты в интерактивном режиме с веб-сервером); Fuzzy Finder или ngram (ищет нечеткие повторы через n-граммы с помощью функции `launch_with_ngram_dup_finder()`); Neuristic выполняет семантический поиск через NLP с использованием Java-инструмента *NLP Clone Detection.jar*. Для нормализации текста перед анализом создаётся промежуточный формат .reformatted, обеспечивая унификацию переноса строк, замену табуляции и конвертацию в кодировку UTF-8. Файлы создаются автоматически при обработке через функцию `save_reformatted_file()` и используются как источник для анализа клонов.

Итогом всех режимов анализа — интерактивные HTML-отчёты, содержащие общий текст для всех повторов текстовых фрагментов, с выделением различий и элементы для навигации по файлу. Рефакторинг позволяет преобразовывать найденные клоны в информационные элементы DRL и словарные записи.

Системные требования включают в себя Python 3.7.x 64-bit с множеством зависимостей (PyQt, PyQtWebEngine, QuaMash, lxml, PyContracts, pygments, NumPy, intervaltree, bottle, pyinterval, chardet, psutil, PyYAML), Pandoc (для конвертации форматов), JRE 10+ (для Neuristic), .NET Framework 4.5 (для работы на Windows) или Mono (для работы Fuzzy Finder на Unix-подобных ОС), Wine (для работы Clone Miner на Unix-подобных ОС). Есть возможность интеграции с DocLine/Eclipse, позволяя возвращать результаты в исходный редактор через Eclipse плагин. При автономном запуске результаты сохраняются

в файлы `.refactored`.

## 1.4. Язык и экосистема Golang

Основой разработки было создание производительного, масштабируемого и надёжного инструмента, поэтому качестве языка реализации был выбран язык Go версии 1.23.

Язык был создан в Google из-за внутренних проблем технологической компании Google. Аппаратное обеспечение, как и программное, огромно, у крупных технологических гигантов существуют миллионы строк кода для ПО, с серверами написанных на C++, большинство остальных компонентов — на Java и Python. Тысячи инженеров работают над кодом, над общим ядром кодовой базы всего ПО, куда каждый день вносятся изменения, порой значительные. Специально разработанная, крупная распределённая система сборки позволяет осуществлять разработку на таких масштабах. Но, несмотря на это, задача всё ещё сложна — критическое ПО запускается на гигантском количестве машин, которые в совокупности рассматриваются как скромное число независимых, объединённых в сеть вычислительных кластеров.

В итоге, процесс вычислений в Google хоть и медленный, а зачастую громоздкий, но всё же чрезвычайно результативный. Поэтому именно в технологической компании появилась нужда в языке, который легко масштабировать и продуктивно использовать. Для решения этих проблем и был стартом проекта языка Go.

Особенности Go, как конкурентность (совокупность независимо выполняемых процессов), через гоурутины (лёгковесный поток (thread), управляемый средой выполнения Go) и каналы (типизированный канал (conduit) для безопасной отправки и получения значений), модули (встроенная в Go система управления версиями наборов связанных пакетов для предсказуемой и надёжной работы с зависимостями), дженерики (способ написания кода, не зависящий от конкретных используемых типов), алгоритм сборки мусора Green Tea (опирается на трёхцветный алгоритм сортировки, позволяющий избежать таких проблем, как

долгий цикл сканирования и лишние процессорные циклы).

## 1.5. Возможности больших языковых моделей в задаче анализа текстов

Большие языковые модели (БЯМ) в сравнении с традиционными алгоритмами NLP, являются более эффективными, особенно в обучении без примеров и обучении с малым количеством примеров. Например, сравнительный анализ предобученных БЯМ показал их высокую результативность в задачах классификации, генерации и суммаризации текста, а в исследовании, при сравнении новостных статей, они превзошли экспертные комментарии, особенно в заданиях, требующих глубокого понимания контекста [17]. В задаче классификации тональности исследователям удалось добиться снижения количества ошибок почти на 30% за счёт ввода структурированных фоновых знаний в процесс подсказки [22].

Для извлечения и суммирования информации, основным стало развитие конвейеров, объединяющих БЯМ с механизмами поиска, так называемых RAG (Retrieval-Augmented Generation, Поисковая расширенная генерация). С их помощью модель точнее отвечает на вопросы, опираясь на актуальные внешние источники, уменьшая вероятность появления галлюцинаций. Например, разработанная система DEREK, использующая гибридный поиск и верификатор ответов, продемонстрировала высокую точность и прослеживаемость при работе с корпоративными документами [32]. Другая похожая разработка, система BRIEF, сжимает данные о множестве документов в плотные релевантные аннотации при ограниченном окне контекста без потери качества ответа [1].

Одним из прорывов в области анализа текстов — использование цепочки рассуждений (Chain-of-Thought, CoT). С помощью этого метода БЯМ имитирует пошаговое решение задач человека, выстраивая логические связи с обоснованием выбранного шага. Развитие этой парадигмы привело к появлению методики динамической рекурсивной CoT, по-

высившей точность рассуждений на сложных бенчмарках, и методики Key-Concept Thinking (рассуждений о ключевых концепциях), направляющая модель на выделение и анализ ключевых концепций задачи [14]. Все перечисленные выше методики с использованием БЯМ позволяют точно извлекать аргументативные структур из текста с использованием техник подсказок и цепочек рассуждений.

Современные обзоры выделяют три ключевых направления развития БЯМ с большим контекстом: получение эффективных моделей, их обучение и развёртывание, комплексная оценка существующих моделей. Для таких анализов предлагается использовать дискурсивные признаки для выявления фактических несоответствий при суммаризации длинных текстов [30] или использование риторической структуры текста для улучшения поиска релевантной информации в длинных документах [2].

Таким образом, БЯМ в задаче анализа текстов используют обширный набор техник для точности результатов. Дальнейшее развитие будет связано с развитием существующих методов, с улучшением интерпретируемости моделей, снижением вычислительных затрат и созданием новых эффективных методов для обработки сверхдлинных последовательностей.

## 2. Требования к фреймворку

### 2.1. Случаи использования и функциональные требования

Основные случаи использования фреймворка включают в себя.

- Пользователь (разработчик/аналитик) передаёт путь к файлу, выбирает режим поиска (automatic, heuristic, openai, ngram), задаёт параметры минимальной длины клонов и минимальной мощности группы, после чего получает структуру результатов со статистикой.
- После анализа пользователь формирует отчёт в одном из форматов: html, json, csv. Отчёты содержат группы дубликатов, их фрагменты, агрегированную статистику, а в HTML/JSON также тепловую карту (heatmap).
- Пользователь передаёт документ в поддерживаемом формате (docbook, docx, odt, rtf, md, txt, html и др.). Если нужен промежуточный этап преобразования, используется конвертер с последующим парсингом.
- Пользователь может создать и подключать собственные модули через реестр модулей для удовлетворения собственных потребностей разработки.

Функциональные требования включают в себя.

- Фреймворк должен принимать основные параметры поиска и путь к файлу.
- Фреймворк должен поддерживать не менее четырёх встроенных алгоритмов поиска: automatic, heuristic, openai, ngram.
- Фреймворк должен извлекать текст из входных документов через парсеры, при необходимости — через конвертацию.



- Фреймворк должен формировать отчёт анализа со списком групп клонов, основной статистикой повторов, метаданными, тепловой картой.
- Фреймворк должен помечать найденные фрагменты координатами в исходном файле анализа.
- Фреймворк должен генерировать отчёты минимум в форматах html, json, csv, в зависимости от выбранного пользователем варианта, по умолчанию формат html.
- Фреймворк должен сохранять отчёты в указанную директорию.
- Фреймворк должен предоставлять интерфейс для расширения через реестр плагинов.

## 2.2. Нефункциональные требования

- Модули фреймворка должны подключаться независимо друг от друга через единый реестр.
- Фреймворк должен корректно работать с распространёнными форматами документации через конвертацию pandoc, при отсутствии возможности конвертации документация должна читаться как текстовые данные.
- Фреймворк должен корректно читать основные входные параметры.
- Фреймворк должен корректно возвращать ошибки анализа.
- Фреймворк должен обрабатывать некорректные случаи анализа (пустые группы, отсутствующие модули, недоступные файлы).
- Алгоритмы и генерация отчётов должны быть пригодны для практического анализа документов среднего и большого объёма.
- Решение должно запускаться в стандартной среде Go.

- Кодовая база должна оставаться поддерживаемой за счёт разделения на модули, интерфейсов компонентов, наличия примеров использования и тестов.
- Открытый API фреймворка должен быть простым для встраивания в сторонний код

## 3. Архитектура фреймворка

### 3.1. Структура фреймворка

Проект представляет собой фреймворк, написанный на языке Go с использованием утилиты randos, основанный на взаимодействии автономных модулей с основой в виде ядра в роли оркестратора. Модули реализуют алгоритмы поиска, парсинг входного файла документации, создание отчётов. При необходимости, при взаимодействии с исходным кодом, представлена возможность реализовать новый модуль и подключить его к ядру через реестр. Стандартное взаимодействие происходит через открытый API при помощи встроенной в Go системы пакетов, позволяя внедрять фреймворк в сторонний код. Общее представление архитектуры фреймворка представлено на Рис. 1.

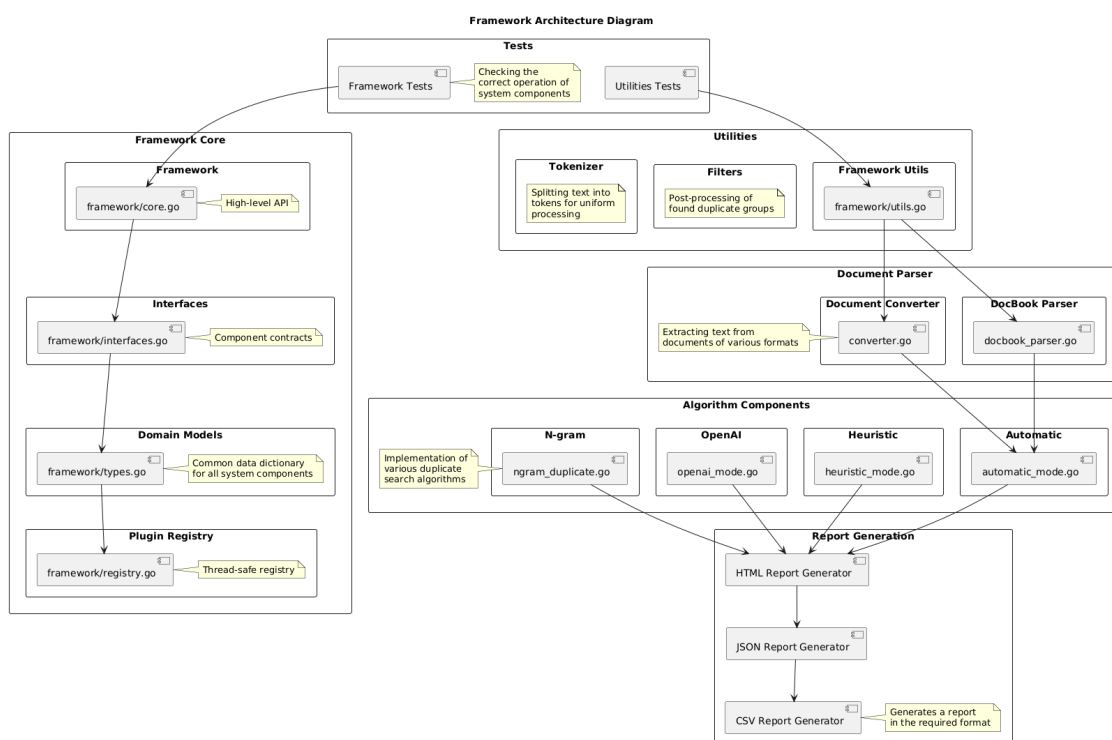


Рис. 1: Архитектура фреймворка

Вход для пользователя расположен в pkg/docline, где находится структура Docline, которая скрывает внутренние детали сборки системы. Через функцию создаётся экземпляр внутренней структуры Framework с его реестр, после происходит регистрация встроенных

компонентов: базовые утилиты ядра, алгоритмы, обработчики документов и репорт-генераторы. За счёт этого у пользователя есть простой сценарий: создать объект, запустить функции `AnalyzeDocument` и `GenerateReport`, без необходимости вручную вызывать необходимые модули. Этот слой является оболочкой для публичного доступа к внутреннему фреймворку: он стабилизирует API, делая взаимодействие простым и предсказуемым.

Центральное ядро находится в `internal/framework`, составляя основу всего фреймворка. Тип `Framework` выполняет роль оркестратора: принимает входные параметры, запускает чтение документа, выбирает алгоритм поиска клонов, формирует данные и передаёт результат в генератор отчёта. При этом `Framework` работает через абстракции: оркестрация отделена от деталей реализации. В этой же директории находится глобальная конфигурация (тип `Config`), определяющая значения по умолчанию для токенизатора, алгоритма поиска, формата отчёта и директории для сохранения отчётов.

Слой ядра для взаимодействия оформлен через интерфейсы в `internal/framework`, где определены роли всех подключаемых компонентов: `CloneFinder`, `DocumentParser`, `DocumentConverter`, `ReportGenerator`, `TextTokenizer`, `Filter`, а также общий тип `Plugin` для выполнения в жизненном цикле. Интерфейсы являются местом добавления новых модулей для проекта, позволяя не трогать основное ядро фреймворка и удобно масштабировать его структуру. То есть, можно реализовать новый алгоритм или новый формат отчёта локально, не затрагивая остальную систему.

Единая модель данных определена в `internal/framework` в виде базовых доменных сущности: `TextFragment` (фрагмент текста с позициями), `CloneGroup` (группа похожих фрагментов), конфиги анализа/фильтрации/сохранения, результат в `AnalysisResult` и `AnalysisStatistics`. Модули из разных пакетов взаимодействуют с помощью единой модели данных и его стандартизированных типов, сохраняя согласованность всего пайплайна. Например, генераторы отчётов могут взаимодействовать с любым алгоритмом поиска клонов, если тот корректно возвращает тип

CloneGroup.

Через тип PluginRegistry в internal/framework управляются подключаемые модули в виде реестра плагинов. Реестр разделён по категориям (finders, parsers, converters, report generators, tokenizers, filters, generic plugins), потокобезопасен и поддерживает явные операции регистрации/получения. Архитектурно реестр представляет собой локатор служб для функционирования фреймворка в контролируемом виде: типы компонентов не смешиваются, имена проверяются на уникальность, а выбор модуля идёт по понятным ключам (имя, расширение, формат).

Базовые встроенные компоненты ядра вынесены отдельно в internal/framework. Туда входят SpaceTokenizer, JaccardSimilarityCalculator, StrictFilter, а также функция пакетной регистрации RegisterBuiltInPlugins, создавая минимальный набор инструментов для работы фреймворка: даже без внешних расширений в фреймворке присутствуют готовые алгоритмы для анализа. При этом встроенные значения не строго заданы в Framework, а также проходят через реестр, то есть при необходимости их можно заменить альтернативными реализациями.

Слой алгоритмов расположен в internal/algorithms. Структурно он состоит из конкретных реализаций режимов и их адаптеров, которые приводят внутренние алгоритмы к интерфейсу CloneFinder: адаптеры устраняют возможные ошибки взаимодействия между интерфейсом взаимодействия алгоритмов и стандартом фреймворка. В результате Framework видит все алгоритмы одинаково и переключает их по имени, не зная, как устроена конкретная реализация.

Слой обработки документов и отчётности сосредоточен в internal/report. По структуре это два подуровня: обработка на вход и формирование отчёта на выходе. Через адаптеры в internal/algorithms они подключаются к интерфейсам DocumentParser, DocumentConverter, ReportGenerator. На этапе чтения документа система использует структуру: прямой парсинг по расширению, затем при необходимости конвертация через pandoc в DocBook, в случае неудачи: конвертация в текстовые данные. На этапе вывода доступно несколько генераторов

(HTML/JSON/CSV), работающих с общей моделью `AnalysisResult`, позволяя взаимодействовать с входными и выходными форматами независимо от выбранного алгоритмов поиска клонов или других модулей фреймворка.

Алгоритм работы конвейера фреймворка не ограничивается поиском повтора и созданием отчёта. После работы алгоритма поиска, фреймворк дополняет фрагменты дополнительными метаданными (например, границами строк), считает статистику и формирует унифицированный объект результата с `metadata/config/stats`: такая работа облегчает взаимодействие человека с отчётом, и, при необходимости, позволяет сразу использовать данные для постобработки другими программами или последующими интеграциями.

### **3.2. Шаблон использования**

Для фреймворка используется конвейер: инициализация, анализ документа, генерация отчета. На практике сначала создается объект `Docline` или `Framework`, куда передается базовый конфиг, затем выполняется анализ входного файла с выбранным режимом поиска дубликатов, на выходе формируется `AnalysisResult` со статистикой и группами клонов. После этого результат передается в генератор отчета, который сохраняет файл. Диаграмма вариантов использования фреймворка представлена на Рис. 2.

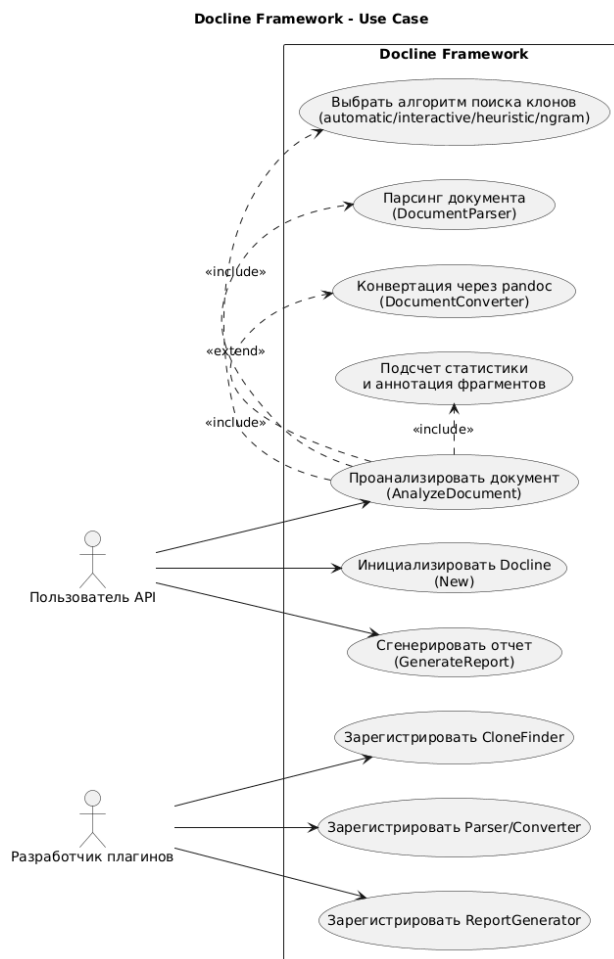


Рис. 2: Диаграмма вариантов использования фреймворка

Стандартный сценарий — использование открытого API с помощью `pkg/docline`: вызов фреймворка и регистрация встроенных плагинов (алгоритмы, парсеры/конвертеры, генераторы отчетов). Пользователь выбирает режим (`automatic`, `heuristic`, `openai`, `ngram`) с параметрами, а система сама выполняет конвейер чтения документа, токенизации, поиска повторов и формирования метаданных: такой шаблон удобен для прикладных задач с минимумом ручной сборки и простой интеграции в сторонний код.

Более гибкий шаблон — работа с исходным кодом и ручная регистрация необходимых модулей: вызов `Framework` и реестра, регистрация нужных модулей и запуск функции `AnalyzeDocument`. Этот шаблон используется при необходимости добавления собственных алгоритмов поиска клонов, форматов отчетов или парсеров/конвертеров.

## 4. Особенности реализации

### 4.1. Система модулей

Система модулей представляет собой архитектуру с плагинами и слоями, четким разделением ответственности между открытым интерфейсом, ядром оркестрации и реализациями конкретных подсистем. Внешний слой находится в `pkg/docline`: это стабильный интерфейс взаимодействия с фреймворком через встроенный в Go функционал внешних пакетных модулей для удобства пользователя, который также скрывает большую часть внутренних деталей сборки. Внутренний слой расположен в `internal/*` и разделён на самостоятельные домены: `internal/framework` (ядро), `internal/algorithms` (алгоритмы поиска клонов), `internal/report` (обработка документов и отчёты). Такое деление разделяет границы зависимостей, формируя модульную платформу, где новые функции добавляются как подключаемые компоненты.

Основным модулем является `internal/framework`, потому что именно он определяет абстракции, через которые взаимодействуют все остальные части. В файле `internal/framework/interfaces.go` зафиксированы интерфейсы `CloneFinder`, `DocumentParser`, `DocumentConverter`, `ReportGenerator`, `TextTokenizer`, `Filter` и общий `Plugin`: алгоритмы и обработчики зависят не друг от друга, а от контрактов ядра. Из-за этого код в `internal/algorithms` не обязан учитывать устройство генераторов отчётов, а модуль отчётности не зависит от конкретной реализации поиска клонов. На уровне типов (`internal/framework/types.go`) ядро также задаёт общий язык данных — `CloneGroup`, `TextFragment`, `AnalysisResult`, `AnalysisStatistics`, `CloneFinderConfig`, `ReportConfig`: модули взаимодействуют через единые структуры, не используя в API различные внутренние форматы друг друга. Второй основной элемент модульной системы — `PluginRegistry` из `internal/framework/registry.go`. Это централизованный контейнер плагинов, внутри которого поддерживаются отдельные реестры по категориям: поисковики, парсеры, конвертеры, генераторы, токенизаторы и фильтры. Реестр потокобезопасен



(из-за `sync.RWMutex`) из-за потенциального параллельного использования фреймворка. Регистрация проверяет уникальность имён и предотвращает дублирование компонентов, а получение идёт по семантике конкретного типа: например, `GetCloneFinder(name)` — по идентификатору, `GetDocumentParser(extension)` — по поддерживаемому расширению, `GetReportGenerator(format)` — по формату вывода. Архитектурно это означает, что добавление нового модуля сводится к реализации интерфейса и регистрации экземпляра; ядро при этом не переписывается.

Тип `Framework` в `internal/framework/core.go` — это модуль-оркестратор, который связывает зарегистрированные плагины в единый пайплайн выполнения. Самостоятельно он не реализует взаимодействия между модулями, а координирует их действия: чтение/преобразование документа, выбор алгоритма поиска, запуск анализа, подсчёт статистики, генерация отчёта. С модульной точки зрения: `Framework` зависит от интерфейсов и реестра, а не от конкретных реализаций. Встроенные базовые плагины ядра (токенайзер по пробелу, сходство Жаккара, строгий фильтр) вынесены отдельно в `internal/framework/builtins.go` и `internal/framework/adapters.go`, позволяя подключать основной набор модулей с помощью одной функции `RegisterBuiltInPlugins` с сохранением расширяемости для пользовательских реализаций.

Модуль алгоритмов `internal/algorithms` подключается к ядру через слой адаптеров (`internal/algorithms/framework_adapters.go`). Алгоритмы могут иметь собственные структуры настроек и внутренние форматы, а адаптер приводит их к одному интерфейсу `framework.CloneFinder`. За счёт этого в реестре все алгоритмы взаимодействуют по общему конфигу (как `CloneFinder`), даже если внутри они сильно различаются по логике и параметрам. Аналогичный приём используется в `internal/report/framework_adapters.go`: парсер `DocBook` и конвертер `pandoc` обёрнуты в интерфейсы ядра `DocumentParser` и `DocumentConverter`, а генераторы отчётов реализуют `ReportGenerator` в `internal/report/report_generators.go`. Модульная система проекта взаимодействует с помощью адаптеров: они отделяют внутреннюю структуру конкретного модуля от общего контракта платформы.

Открытый API `pkg/docline` выступает как интерфейс над этой системой. В `pkg/docline/docline.go` конструктор `New` поднимает тип `Framework`, получает реестр и регистрирует встроенные модули: базовые компоненты ядра, алгоритмы, обработчики документов, генераторы отчётов. Для пользователя это выглядит как готовая сборка, которую можно подключить как модуль Go, но внутри сохраняется плагиновая природа. При этом пакет добавляет ещё один уровень модульности — типобезопасные режимные конфиги (`pkg/docline/mode_configs.go`), которые мапятся в универсальный `CloneFinderConfig`. Это отделяет внешний UX-конфигуратор от внутреннего интерфейса параметров (`CustomParams`), снижающая связанность между пользователем API и конкретными параметрами внутренних конфигов.

Система модулей построена как совместный конвейер ядра контрактов, реестра, адаптеров и фасада: ядро фиксирует возможности модуля, реестр управляет жизненным циклом и поиском модулей, адаптеры согласуют разные реализации с единым интерфейсом, а интерфейс создаёт удобный вход без необходимости вручную собирать граф зависимостей. Из-за этого фреймворк имеет возможность расширяться горизонтально: новые алгоритмы, форматы документов и типы отчётов можно добавлять независимо друг от друга, сохраняя стабильный открытый API.

## 4.2. Встроенные алгоритмы

Встроенный набор алгоритмов включает режимы поиска клонов: `automatic`, `heuristic`, `ngram` и `openai`, подключаемые через адаптеры к единому интерфейсу взаимодействия `CloneFinder`. С точки зрения ядра все алгоритмы выглядят одинаково: каждый умеет принять нормализованный текст и конфиг, вернуть группы клонов (структура `CloneGroup`) и предоставить свой идентификатор/описание. Связь между ядром и конкретными реализациями алгоритма проходит через адаптеры в `internal/algorithms/framework_adapters.go`. Именно там регистрируются встроенные алгоритмы (`automatic`, `heuristic`, `openai`, `ngram`),

а также выполняется преобразование универсального конфига через `framework.CloneFinderConfig` в специфические настройки каждого режима.

Алгоритм **automatic** реализован в `algorithms/automatic_mode.go` и рассчитан на максимально автоматический детерминированный поиск повторов. Базовая логика — поиск скользящих окон фиксированной длины через параметр `MinCloneLength` по последовательности токенов. Сначала строится частотная карта окон, затем собираются позиции только для тех окон, которые встречаются минимум дважды. Это двух-проходная схема, которая уменьшает лишние аллокации и делает поведение предсказуемым на больших текстах. После первичного построения групп выполняется дополнительное укрупнение: группы с похожими архетипами сливаются по сходству Жаккара (по умолчанию порог 0.9), а затем, при включённом `StrictFilter`, применяются фильтры на минимальную длину архетипа и удаление пересекающихся фрагментов. В этом же режиме поддерживается предварительное преобразование текста в DRL-подобную нормализованную форму (`convertToDRL`): приводится регистр, убирается пунктуация/символы, преобразовываются пробелы. Режим поиска `automatic` находит устойчивые повторяющиеся окна без пользовательской настройки на каждом шаге.

Алгоритм **heuristic** из `internal/algorithms/heuristic_mode.go` построен на основе эвристического алгоритма, использующего коэффициент Жаккара для поиска схожих повторов текста. Функция `splitTextIntoSentenceSegments` делит текст на фрагменты примерно размером с предложение. Каждый такой сегмент очищается от символов пунктуации, приводится к нижнему регистру, разбивается на токены и из них собирается частотный словарь. Затем для всех пар сегментов вычисляется коэффициент Жаккара и, если значение превышает значение `CustomParams`, то сегменты объединяются в компоненты связности. После этого из каждой компоненты отбираются сегменты, чья длина в токенах не меньше минимального значения `MinCloneLength`. При достаточном количестве таких сегментов `MinGroupPower` формируется группа клонов, содержащая эталонный фрагмент, список фрагментов

текста, их позиции в исходном документе (в токенах) и мощность группы.

Алгоритм **ngram** реализован в `internal/algorithms/ngram_duplicate.go` и использует n-граммное представление текста. Он строит n-граммные карты (`BuildNGramMap`) для частей текста и измеряет сходство через отношение пересечения к объединению множеств n-грамм (`CalculateNGramSimilarity`). Дубликаты фиксируются, когда похожесть превышает порог, вычисляемый из `MaxFuzzySlider`. Важная особенность: адаптер сначала делит входной текст на фрагменты по окончаниям предложений (`splitTextIntoParts` в `framework_adapters.go`), после чего сравнение идёт уже между этими частями. Функция `Framework.AnalyzeDocument` не знает внутренних деталей режима: она читает документ, получает алгоритм по имени из реестра, вызывает `FindClones`, затем дополняет фрагменты номерами строк и считает сводную статистику, из-за чего все алгоритмы выдают результаты в единой модели (`CloneGroup`, `TextFragment`) и в стандартном виде попадают в отчёты HTML/JSON/CSV. Для пользователя это означает единый интерфейс запуска при разной внутренней математике.

Алгоритм **openai** реализован как поиск нечётких повторов с использованием больших языковых моделей и интегрирован в слой `CloneFinder` по той же схеме, что и остальные встроенные режимы: он принимает исходный текст, формирует запрос в OpenAI-совместимый API с промптом вида "Найди все нечёткие повторы в тексте", который при необходимости можно изменить, затем преобразует JSON-ответ модели в стандартные для проекта `CloneGroup/TextFragment` с позициями и архетипами. Режим поддерживает два провайдера в одном интерфейсе: облачный OpenAI API (с `api_key` и доступной БЯМ) и локальный Ollama (через `base_url`, без обязательной авторизации), поэтому может работать как в production-интеграции, так и в полностью офлайн-сценарии. Для стабильности результата в алгоритме предусмотрены строгий системный промпт для точности, валидация структуры групп (минимум 2 фрагмента) и аккуратная постобработка ответа перед маппингом во внутренний формат фреймворка.

Краткое описание режимов поиска повторов: `automatic` — строгий и конвейерный поиск устойчивых повторов, `rugkshyeshc` — поиск повторов по эвристическому алгоритму, `ngram` — нечёткое сравнение сегментов по n-граммам, `orepai` — поиск с помощью БЯМ. Все они уже приведены к общему конвейеру взаимодействия и могут комбинироваться с одной и той же подсистемой обработки документов и генерации отчётов.

### 4.3. Обработка документов

Основные компоненты модуля находится в `internal/framework/core.go`. Метод `readDocument` управляет маршрутизацией документа: сначала определяет расширение файла, затем пытается найти подходящий `DocumentParser` через реестр плагинов. Если парсер под это расширение зарегистрирован, файл открывается как поток (с помощью `io.Reader`), парсер возвращает набор текстовых сегментов, ядро объединяет эти сегменты в единый анализируемый текст. Если прямого парсера для расширения нет, `readDocument` переключается на конвертацию. Через реестр задействуется интерфейс `DocumentConverter` (используется сторонняя утилита `pandoc`), после чего вызывается проверка `IsConversionNeeded(filePath)`. При положительном ответе запускается преобразование входного файла в DocBook XML (.xml), затем уже к временному XML-файлу применяется парсер в DocBook. Модуль конвертации не дублирует логику извлечения текста, а подготавливает файл к уже существующему парсеру. Конвертер реализован в `internal/report/converter.go` как `DocumentConverter`. Он хранит карту поддерживаемых входных форматов (.doc, .docx, .odt, .rtf, .md, .txt, .html, .htm) и поддерживаемых целевых DocBook-форматов (.xml, .dbk, .docbook). Внутри используется утилита `pandoc` через `exec.Command`: выбирается исходный формат по расширению (`getPandocFormat`), затем выполняется преобразование к формату `docbook`. Конвертация пишет временный файл в системную временную директорию, а в ядре после чтения этот временный файл удаляется

(defer os.Remove(tempPath)). Цикл временных артефактов встроен в пайплайн и не перекладывается на код.

Парсинг DocBook реализован в `internal/report/docbook_parser.go` (`DocBookParser`) и использует XML-декодер со стандартным параметром `Strict=false`, делая разбор устойчивее к ошибкам в XML. Дополнительно на уровне декодера задана таблица сущностей (`ndash`, `mdash`, `nbsp`, и т.д.) для корректной обработки типичных именований сущности. Парсер рекурсивно проходит дерево элементов и извлекает текст только из заданного набора тегов (`para`, `section`, `chapter`, `title`, `simpara`, `note`, `warning`, `important`, `tip`), уменьшая шум и держа фокус на содержательных текстовых узлах, а не на всей XML-структуре. При этом набор тегов расширяемый: есть методы добавления/удаления типов элементов из извлечения. В дополнение к парсеру DocBook, в фреймворке реализован отдельный парсер для формата DRL — `DRLParserAdapter` в `internal/report/drl_parser_adapter.go`. Он также работает как XML-поточковый разбор, но в ином, более простом режиме: собирает все непустые `CharData`-узлы и объединяет их в один сегмент. Назначение этого адаптера — не потерять текст внутри специфичных DRL-конструкций, где полезный контент может быть распределён по различным элементам. Из-за регистрации этого парсера в `RegisterDocumentPlugins`, формат `.drl` автоматически встраивается в общий путь обработки.

Слой адаптеров `internal/report/framework_adapters.go` связывает конкретные реализации с интерфейсами ядра. `DocBookParserAdapter` реализует `framework.DocumentParser`, `PandocConverterAdapter` — `framework.DocumentConverter`, а функция `RegisterDocumentPlugins` регистрирует оба парсера DRL и XML в `PluginRegistry`: ядро не знает о классах `DocBookParser` и `DocumentConverter` напрямую; оно работает через интерфейсы и реестр. В архитектурном смысле обработка документов полностью реализована с помощью плагинов: можно добавить новый парсер/конвертер, не меняя тип `Framework`, для этого нужно реализовать интерфейс и зарегистрировать модуль. Если ни специализированный парсер, ни конвертация не могут преобразовать текст для анализа, применяется резервный способ: файл читается как

текстовые данные через `os.ReadFile`. Несмотря на ненадёжность со снижением качества анализа, является компромиссом для успешной работы пайплайна.

После того как текст получен и клоны найдены, обработка документа продолжается в виде пост-обогащения фрагментов: через `core.go` вызывается `annotateFragmentsWithLineNumbers`, по токенизированному тексту вычисляются номера строк начала/конца для каждого фрагмента (`source_line_start`, `source_line_end`), не меняя сам алгоритм поиска, но существенно улучшая интерпретируемость результата и качество отчётов, связывая найденные дубликаты с исходным текстовым контекстом.

## 4.4. Создание отчётов

Вход модуля находится в `internal/framework/core.go`, в методе `GenerateReport`, где получает уже сформированный `AnalysisResult`, выбирает генератор по формату через `PluginRegistry` и передаёт ему унифицированные данные — группы клонов и `ReportConfig`. На стороне ядра перед вызовом генератора выполняется подготовка контекста отчёта: `GenerateReport` создаёт отдельную копию данных из результата анализа и добавляет туда статистику данных, чтобы генераторы могли использовать параметры анализа, не изменяя исходный `AnalysisResult`. Затем формируется `ReportConfig` с заголовком, путём исходного файла, набором настроек и директорией вывода для сохранения файлов отчётов.

Встроенные генераторы отчётов реализованы в `internal/report/report_generators.go` и подключаются через интерфейс `RegisterReportGenerators`. Во встроенном наборе есть три формата: HTML, JSON и CSV. Все они реализуют интерфейс `ReportGenerator`, обязаны содержать в себе данные `Name`(название), `Format`(расширение файла) и `Generate` (сам файл). HTML-генератор создаёт структуру страницы с заголовком, источником, количеством групп и таблицей, где каждая группа содержит `power`, `archetype` и список фрагментов.

Отдельно встроена визуализация покрытия документа — тепловая карта по токенам; для её построения сначала определяется общее число токенов (из данных `total_tokens`, а при отсутствии — через максимальное значение `EndPos` по фрагментам), затем вычисляется распределение покрытия по фиксированному числу бинов: в HTML это рисуется как полоса интенсивностей с градиентом, что даёт чёткий визуальный сигнал о количестве повторов по частям документа. Генератор использует данные о строках (`source_line_start/source_line_end`), добавляя к фрагментам префиксы вида `Lx` или `Lx-y`, где `x` и `y` являются номерами строк, для удобства поиска необходимых фрагментов.

JSON-генератор предназначен для использования внутри пайплайна и интеграций в отчёты, формируя структурированный набор данных с заголовком, исходным файлом, настройками, массивом групп, статистикой, тепловой картой и общим числом токенов. После данные сериализуются через функцию `json.MarshalIndent`. CSV-генератор реализует более компактный табличный отчёт, ориентированный на краткие повторяющиеся фрагменты и сценарии быстрой офисной аналитики; создаёт файл с разделителем, добавляя UTF-8 BOM для корректного открытия в Excel, и пишет записи в формате "число токенов / количество повторов / текст". В генераторе встроены прикладные фильтры: ограничение длины фрагмента (`MaxTokens`) и минимальная частота (`MinOccurs`). Общее между генераторами — подготовка файловой среды перед записью: каждый генератор создаёт директорию вывода через `os.MkdirAll`, снижая вероятность ошибок на этапе сохранения. Далее каждый формат записывает данные в свой целевой файл (`os.WriteFile` для HTML/JSON, `csv.Writer` для CSV). Функции типа `buildHeatmap`, `totalTokensFromSettings`, `statsFromSettings`, `intFromMetadata` и `overlapLen` работают как внутренний слой подготовки данных для нескольких генераторов сразу, уменьшая потенциальное дублирование и делая поведение форматов согласованным. Например, для HTML и JSON используется одинаковая логика создания тепловой карты.

С архитектурной точки зрения, подсистема отчётов реализована как



расширяемый слой поверх унифицированного результата анализа. Она отделена от поиска клонов, подключается через реестр, использует стабильный ReportConfig и поддерживает разные потребности: визуальный отчёт для чтения (HTML), структурированный экспорт для интеграций (JSON) и компактный табличный срез для прикладной обработки (CSV). Это упрощает добавление нового формата к изменению в ReportGenerator и его регистрации, без изменения ядра фреймворка или существующих алгоритмов.

## 4.5. Открытый API

Открытый API реализован через пакет pkg/docline, представляя собой интерфейс над внутренним фреймворком. Внешний пользователь, используя встроенную в Go установку сторонних пакетов, работает с небольшим, стабильным набором сущностей и методов, а вся сложность — регистрация плагинов, маршрутизация к алгоритмам, обработка форматов документов и интеграция генераторов отчётов — остаётся внутри. По дизайну API ориентирован на беспроблемное и базовое использование фреймвока, при этом сохраняя достаточную гибкость конфигурации для разных режимов анализа.

Центральная сущность API — тип Docline из pkg/docline/docline.go. Экземпляр создаётся через New(cfg \*Config), где Config задаёт стандартные конфигурации работы: директорию результатов, формат отчёта по умолчанию, токенизатор по умолчанию и стандартный поиск повторов. При создании Docline внутри поднимается интерфейс internal/framework.Framework, затем автоматически регистрируются встроенные компоненты ядра, алгоритмы, обработчики документов и генераторы отчётов. Для клиента не нужно вручную вызывать регистрацию каждого модуля или собирать пайплайн.

Типобезопасные конфиги находятся в pkg/docline/mode\_configs.go и покрывают встроенные режимы: AutomaticConfig, HeuristicConfig, NgramConfig, OpenAIConfig. Каждый из них реализует два уровня контракта: публичный (FinderType) и внутренний (toInternal), позво-

для держать пользовательский API чистым и понятным, а внутренние специфичные параметры аккуратно прокидывать через CustomParams только там, где это нужно. Например, для режима automatic передаются флаги вроде `convert_to_drl` и `strict_filter`, для `heuristic` — `similarity_threshold`, для режима `ngram` — параметры поиска нечётких повторов и путь файла, для режима `openai` — промт и название БЯМ. Внешний код при этом остаётся читаемым и структурированным, без ручного управления внутренними параметрами. В случае необходимости работы через специальные конфиги, реализован метод `AnalyzeDocumentWithConfig`, который принимает более общий `CloneFinderConfig` (присутствует в публичном пакете). Это альтернативный путь для случаев, когда нужно работать ближе к внутренней модели и вручную формировать `CustomParam`, давая открытому API две стратегии: безопасный режим через специализированные конфиги и универсальный режим через общий конфиг.

Вторая ключевая функция API — `GenerateReport(result, format, outputPath)`. Она не требует заново передавать исходный документ или параметры анализа, а работает с уже полученным `AnalysisResult`: сначала анализ, потом презентация результата: один и тот же результат анализа можно экспортировать в несколько форматов (например, HTML для просмотра и JSON для автоматической обработки) без повторного запуска алгоритма.

Возвращаемый из анализа `AnalysisResult` принадлежит внутреннему пакету `internal/framework`, но через открытый API он становится основным транспортом данных между фазами пайплайна. В нём есть группы клонов, статистика, метаданные и конфиг, с которым проводился запуск. Для пользователя это общее место с основными данными анализа, пригодный для генерации стандартных отчётов и для собственной постобработки в коде

Основным преимуществом открытого API является компактность, предсказуемость, расширяемость и удобство использования. Основной сценарий использует три основных функции: `New`, `AnalyzeDocument`, `GenerateReport`. Каждый режим анализа повторов имеет свои пара-

метры, с проверками на совместимость, при необходимости используя специальные конфиги. Внутренние алгоритмы и форматы могут развиваться, а внешний способ вызова остаётся стабильным. Поэтому pkg/docline даёт хороший баланс между простотой использования Go и контролем для более сложных интеграционных сценариев.

## 5. Тестирование и апробация

### 5.1. Тестирование фреймворка

Для тестирования корректности функционирования фреймворка был сформирован набор тестов, сочетающий модульные и интеграционные сценарии конвейера ”чтение, анализ, отчёт”.

Подсистема преобразования форматов проверяет необходимость конвертации входного файла (базовые случаи и табличная параметризация) и выполняет операцию удаления временного артефакта. Для адаптера конвертера дополнительно тестируется отказ при неподдерживаемом выходном формате.

Реестр плагинов тестируется на корректность регистрации и извлечения реализаций по названию, включая вызовы к отсутствующим компонентам, обработку дублирующихся регистраций, жизненный цикл и отмену регистрации.

Парсинг DocBook охватывает соответствие стартового набора извлекаемых элементов спецификации, изменяемость конфигурации элементов, извлечение сегментов из XML, обработку синтаксически некорректного входа и регрессионные прогоны на тестовых файлах. Корректность работы адаптера интерфейса фреймворка и адаптера для YAML также проверяются.

Парсер DRL верифицируется по критерию извлечения содержательного текста при исключении служебной разметки. Для открытого API фреймворка проверяются предусловия и согласование идентификатора режима с конфигурацией.

Интеграционный уровень включает сквозной анализ при регистрации встроенных плагинов на синтетическом DocBook, параметрический прогон встроенных локальных поисковиков на YAML, сценарии маршрутизации по расширению с использованием тестового дублёра конвертера (вместо внешнего pandoc). Генерация отчётов проверяется по наличию структуры HTML, согласованности форматов с валидацией JSON и структуры CSV, устойчивости к неполным метаданным.

Непрерывная интеграция реализована средствами GitHub Actions: выполняются форматирование выбранных каталогов, статический анализ и целевые вызовы через встроенные в Go инструменты, в том числе go test, с детектором гонок -race, и сбор профиля покрытия с выгрузкой артефакта. Дополнительно задаётся выполнение кода на нескольких ОС версии Go 1.23.x для снижения рисков из-за различий платформенных сред.

Результаты тестирования можно найти в [Github Actions фреймворка](#).

## 5.2. Апробация фреймворка

Для апробации разработанного фреймворка был проведён анализ документаций известных разработок с помощью встроенных алгоритмов поиска повторов. Анализируемая документация и алгоритмы представлены в Таблице 1.

Таблица 1: Документация с основными данными и алгоритм анализа для апробации

| Проект       | Тип проекта                      | Тип документации         | Объём, КБ | Алгоритм       |
|--------------|----------------------------------|--------------------------|-----------|----------------|
| Linux kernel | Ядро операционной системы        | Руководство пользователя | 892       | ИИ             |
| DocBook      | Язык разметки                    | Руководство пользователя | 702       | Эвристический  |
| OpenAI API   | Интерфейс работы с БЯМ           | API документация         | 2790      | Автоматический |
| Ollama       | Платформа локального запуска БЯМ | API документация         | 56        | Ngram          |

Анализ проводился на едином оборудовании с характеристиками Windows 11, 16 ГБ ОЗУ, процессор Intel Core i5-12400.

Файлы документации и результаты анализа доступны в репозитории [PavelMkr/docline-new](#)

# Заключение

В ходе данной работы были получены результаты, перечисленные ниже.

- Проанализированы алгоритмы поиска повторов в технической документации (коэффициент Жаккара, сходство Левенштейна, алгоритм Рабина–Карпа), инструмент Duplicate Finder; проанализированы перспективные подходы поиска смысловых повторов с использованием больших языковых моделей.
- Сформулированы требования к фреймворку для работы с повторами: 8 функциональных и 9 нефункциональных требований на основе анализа предыдущей реализации.
- На основе требований была выработана архитектура фреймворка с использованием экосистемы языка Go и утилиты pandoc.
- Выполнена реализация основных модулей фреймворка на языке Go: встроенные алгоритмы анализа, обработка документов и создание отчётов, открытый API.
- Проведено испытание по методологии GQM разработанного фреймворка существующих алгоритмов работы с повторами и новых модулей поиска повторов на основе БЯМ с тестированием новой реализации в разных операционных системах через модульные тесты; выполнена апробация созданного инструмента на базе документации существующих программных проектов (Linux kernel, DocBook, OpenAI API, Ollama).

Исходный код можно найти в репозитории [PavelMkr/docline-new](https://github.com/PavelMkr/docline-new).

## Список литературы

- [1] Li Yuankai, Gu Jia-Chen, Wu Di et al. BRIEF: Bridging Retrieval and Inference for Multi-hop Reasoning via Compression. — 2025.
- [2] Chen Huiyao, Yang Yi, Li Yinghui et al. Beyond Chunking: Discourse-Aware Hierarchical Retrieval for Long Document Question Answering. — arXiv preprint. — 2026. — URL: <https://arxiv.org/abs/2506.06313>.
- [3] C.H.G Wright. Technical writing tools for engineers and scientist // Computing in Science and Engineering. — 2010. — Vol. 12, no. 5 p. 98-103.
- [4] Cordy James R., Roy Chanchal Kumar. The NiCad Clone Detector // 2011 IEEE 19th International Conference on Program Comprehension. — 2011. — P. 219–220. — URL: <https://api.semanticscholar.org/CorpusID:2991109>.
- [5] D.L. Parnas. Precise Documentation: The Key to Better Software // Springer. — 2011.
- [6] Detecting the origin of text segments efficiently / Ossama Abdel Hamid, Behshad Behzadi, Stefan Christoph, Monika Henzinger // The Web Conference. — 2009. — URL: <https://api.semanticscholar.org/CorpusID:783953>.
- [7] Dmitry V. Koznov Dmitry V. Luciv George A. Chernishev. Duplicate management in software documentation maintenance // CEUR Workshop Proceedings. — 2017. — Vol. 1989.
- [8] DocLine. — URL: <https://docline.github.io/docline/index.ru> (дата обращения: 15 октября 2025 г.).
- [9] DocReuse: Documentation duplication detector based on Colibri-Java library. — URL: <https://github.com/docreuse/docreuse> (дата обращения: 28 апреля 2026 г.).

- [10] Holmes R. Walker R.J. Systematizing Pragmatic Software Reuse // ACM Transactions on Software Engineering and Methodology. — 2013. — Vol. 21, no. 4 p. 44.
- [11] Juergens Elmar, Deissenboeck Florian, Hummel Benjamin. [CloneDetective - A workbench for clone detection research](#) // Proceedings of the 31st International Conference on Software Engineering. — ICSE '09. — USA : IEEE Computer Society, 2009. — P. 603–606. — URL: <https://doi.org/10.1109/ICSE.2009.5070566>.
- [12] K. Czarnecki. Reuse and Evolution with Generative Techniques // Proceedings of the IEEE/ACM International Conference on Automated Software Engineering. — 2007. — no. p. 575.
- [13] Karp Richard M., Rabin Michael O. Efficient randomized pattern-matching algorithms // [IBM J. Res. Dev.](#) — 1987. — Vol. 31, no. 2. — P. 249–260. — URL: <https://doi.org/10.1147/rd.312.0249>.
- [14] Key-concept thinking prompting for improved reasoning in large language models / Minghua Tang, Chen Bian, Liming Yang, Xueling Zhong // [Neurocomputing](#). — 2025. — Vol. 656. — P. 130986. — URL: <https://www.sciencedirect.com/science/article/pii/S0925231225016583>.
- [15] Koznov D.V. Detecting Near Duplicates in Software Documentation // ArXiv EPrint. — 2017.
- [16] Koznov D.V. Romanovsky K.Yu. DocLine: A method for software product lines documentation development // Programming and Computer Software. — 2008. — Vol. 34, no. 4 p. 216-224.
- [17] LLMs outperform outsourced human coders on complex textual analysis / Victor J. Bermejo, Andres Gago, Raúl H. Gálvez, Noelia Harari // [Scientific Reports](#). — 2025. — Vol. 15, no. 1.
- [18] Lingo4G. — URL: <https://carrotsearch.com/lingo4g/> (дата обращения: 19 октября 2025 г.).



- [19] Luciv D. Duplicate finder toolkit // Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings. — 2018.
- [20] M. Horie. Tool Support for Crosscutting Concerns of API Documentation // Proceedings of the 9th International Conference on AspectOriented Software Developmen. — 2010.
- [21] Manber Udi. FINDING SIMILAR FILES IN A LARGE FILE SYSTEM // Department of Computer Science, University of Arizona. — 1994. — URL: <https://www.cs.princeton.edu/courses/archive/spr05/cos598E/bib/manber94finding.pdf> (дата обращения: 1 ноября 2025 г.).
- [22] [Modeling Background Knowledge with Frame Semantics for Fine-grained Sentiment Classification](#) / Muhammad Okky Ibrohim, Valerio Basile, Danilo Croce et al. // Proceedings of the 2nd Workshop on Analogical Abstraction in Cognition, Perception, and Language (Analogy-Angle II). — Association for Computational Linguistics, 2025. — P. 22–36. — URL: <https://aclanthology.org/2025.analogyangle-1.3/>.
- [23] Multidisciplinary Instruction with the Natural Language Toolkit / Steven Bird, Ewan Klein, Edward Loper, Jason Baldridge // Proceedings of the Third Workshop on Issues in Teaching Computational Linguistics. — Association for Computational Linguistics, 2008. — P. 62–70. — URL: <https://aclanthology.org/W08-0208/>.
- [24] Near-duplicates and shingling. — URL: <https://nlp.stanford.edu/IR-book/html/htmledition/near-duplicates-and-shingling-1.html> (дата обращения: 19 октября 2025 г.).
- [25] Oumaziz M. A. Documentation Reuse: Hot or Not? An Empirical Study // Springer International Publishing. — 2017.

- [26] Romanovsky K. Koznov D. Minchin L. Refactoring the Documentation of Software Product Lines // Lecture Notes in Computer Science. — 2011. — Vol. 4980, no. p. 158-170.
- [27] Т.Т. Tanimoto. Двоичные коды с исправлением выпадений, вставок и замещений символов // IBM Internal Report. — 1967.
- [28] Trados Studio. — URL: <https://www.trados.com/product/studio/> (дата обращения: 19 октября 2025 г.).
- [29] Ukkonen Esko. Algorithms for Approximate String Matching // Inf. Control. — 1985. — Vol. 64. — P. 100–118. — URL: <https://api.semanticscholar.org/CorpusID:205886218>.
- [30] Zhong Yang, Litman Diane. [Discourse-Driven Evaluation: Unveiling Factual Inconsistency in Long Document Summarization](#) // Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). — Association for Computational Linguistics, 2025. — P. 2050–2073. — URL: <https://aclanthology.org/2025.naacl-long.103/>.
- [31] dupeGuru. — URL: <https://dupeguru.voltaicideas.net/> (дата обращения: 15 октября 2025 г.).
- [32] Shi Isaac, Li Zeyuan, Liu Fan et al. eSapiens’s DEREK Module: Deep Extraction & Reasoning Engine for Knowledge with LLMs. — arXiv preprint. — 2025. — arXiv:2507.15863. URL: <https://arxiv.org/abs/2507.15863>.
- [33] pldoctoolkit repository first commit. — URL: <https://github.com/spbu-se/pldoctoolkit/tree/8e371c9d886e43da2fc50ed1493226c40d94b3b3> (дата обращения: 15 октября 2025 г.).

- [34] А.А. Шалыто. Новая инициатива в программировании. Движение за открытую программную документацию // PC Week. — 2003. — no. 40 p. 38-42.
- [35] Антиплагиат.ру. — URL: <https://antiplagiat.ru/about/> (дата обращения: 19 октября 2025 г.).
- [36] Гращенко Л. А. Романишин Г. В. Опыт автоматизированного анализа повторов в научных текстах // Новые информационные технологии в автоматизированных системах. — 2015. — Vol. 18. — P. 582–590.
- [37] Д.В. Луцев. Задача поиска нечетких повторов при организации повторного использования документации // Программирование. — 2016. — no. 4 p.39-49.
- [38] Д.В.Кознов. Поиск клонов при рефакторинге технической документации // Компьютерные инструменты в образовании. — 2012. — no. 4 p. 30-40.
- [39] И. Левенштейн В. Двоичные коды с исправлением выпадений, вставок и замещений символов // Доклады Академий Наук СССР. — 1965. — P. 845–848.